

# Marcov Chain Monte Carlo (MCMC) Direct Simulation Monte Carlo (DSMC)

Kristina Kislyakova

University of Vienna

# Markov Chain Monte Carlo

# Outline

- Introduction to Bayesian statistics and inference, Markov processes and chains
- Introduction to Markov Chain Monte Carlo (MCMC)
- Sampling from PDFs, random walk, Metropolis-Hastings algorithm and Gibbs sampling
- Assignment

Literature:

**van Ravenzwaaij et al.** Psychon Bull Rev (2018). A simple introduction to Markov Chain Monte-Carlo sampling (a short, but very good introduction)

**Cameron Davidson-Pilon**, Probabilistic Programming and Bayesian Methods for Hackers, <https://github.com/CamDavidsonPilon/Probabilistic-Programming-and-Bayesian-Methods-for-Hackers>

# Marcov Chain Monte Carlo (MCMC)

The main idea of MCMC: use Bayesian inference to estimate posterior distribution.

In English: sampling from a probability density function (PDF) given the data and a hypothesis about the model and fitting some model parameters to it. **NB:** an educated preliminary guess about **the model** (the general shape of the function) is necessary. A good guess about **the first starting point** is desirable.

# Marcov Chain Monte Carlo (MCMC)

The main idea of MCMC: use Bayesian inference to estimate posterior distribution.

In English: sampling from a probability density function (PDF) given the data and a hypothesis about the model and fitting some model parameters to it. **NB:** an educated preliminary guess about **the model** (the general shape of the function) is necessary. A good guess about **the first starting point** is desirable.

Why not just use the statistical analytical expressions? You **don't know** the exact height of your PDF at any point. MCMC samples from the distribution by “wandering around” on that distribution so that the amount of time spent in each location is proportional to the height of the distribution.

# Possible applications of MCMC

MCMC methods are designed for sampling **multi-dimensional integrals** and allows to create large hierarchical models that allow integration over hundreds or even thousands of parameters

- Computational physics
- Astrophysics (stellar physics, planetary physics...)
- Computational biology
- Linguistics
- Medical research and behavioral research
- ...and more!

# Bayes' theorem

$$P(A|B) = \frac{P(B|A)P(A)}{P(B)}$$

$A$  and  $B$  are some events and the probability of  $P(B) \neq 0$ .

- $P(B|A)$  is the **conditional probability**: the likelihood of event  $A$  occurring given that  $B$  is true
- $P(A|B)$  is another **conditional probability**: the likelihood  $B$  occurring knowing that  $A$  is true
- $P(A)$  and  $P(B)$  are **marginal probabilities**, i.e. the probabilities of observing  $A$  and  $B$  independent of each other

## Bayes' theorem: example

A medical example: let's assume that 1% of the population has lung cancer and 20% of the population are smokers. Also, suppose that 70% of all people who have lung cancer are smokers. What is the probability of a person having lung cancer given that the person is a smoker ( $P(A|B)$ )?

## Bayes' theorem: example

**A medical example:** let's assume that 1% of the population has lung cancer and 20% of the population are smokers. Also, suppose that 70% of all people who have lung cancer are smokers. What is the probability of a person having lung cancer given that the person is a smoker ( $P(A|B)$ )?

- $P(A)$  - the probability that a person has lung cancer (0.01)
- $P(B)$  - the probability that a person smokes (0.2)
- $P(B|A)$  - the probability how many people who have lung cancer are smokers (0.7)

$$P(A|B) = \frac{P(B|A)P(A)}{P(B)} = \frac{0.7 \times 0.01}{0.2} = 0.035 \text{ (3.5\%)}$$

# Prior and posterior probabilities

- The **prior probability** is the probability distribution you have *before* you have sampled your data (in the previous example,  $P(A)$ )
- The **posterior probability** is the probability distribution you have *after* you have sampled your data (the probability of your model given your data  $P(A|B)$ , or the probability of your data given your model,  $P(B|A)$ )

# Prior and posterior probabilities

- The **prior probability** is the probability distribution you have *before* you have sampled your data (in the previous example,  $P(A)$ )
- The **posterior probability** is the probability distribution you have *after* you have sampled your data (the probability of your model given your data  $P(A|B)$ , or the probability of your data given your model,  $P(B|A)$ )

The prior probability density is often taken to be **constant** over the allowed region (we will make this assumption too in our examples).

# Probability and Likelihood

- **Probability** describes a function of the outcome given a fixed parameter value. For instance: a coin is flipped 10 times and it is a fair coin, what is the probability of it landing heads-up every time?
- **Likelihood** describes a function of a parameter given an outcome. For instance: a coin is flipped 10 times and it has landed heads-up 10 times, what is the likelihood that the coin is fair?

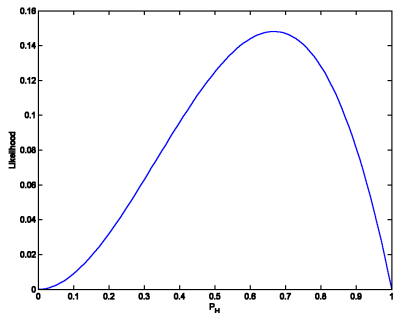
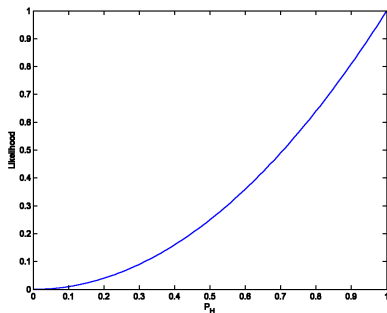
# Probability and Likelihood

- **Probability** describes a function of the outcome given a fixed parameter value. For instance: a coin is flipped 10 times and it is a fair coin, what is the probability of it landing heads-up every time?
- **Likelihood** describes a function of a parameter given an outcome. For instance: a coin is flipped 10 times and it has landed heads-up 10 times, what is the likelihood that the coin is fair?

## Important to know:

- Maximizing the probability density at **your dataset** amounts to **maximizing the likelihood** of the specific observation (see Wikipedia and math books for the mathematical justification)
- All probabilities should add up to unity (**the PDF should be normalized**), but the likelihood is not necessarily normalized

## Likelihood can be calculated and plotted



$P_H$  is the probability that a coin lands “heads up” when tossed (the “fairness” of the coin). **Left panel**: the likelihood that for the probability of a coin landing heads-up (without prior knowledge of the coin’s fairness), given that we have observed HH. **Right panel**: same, but we have observed HHT.

# Bayesian and Frequentist statistics

The difference lies in the interpretation of the concept of probability.

In the frequentists' approach, the probabilities are not attached to hypotheses or to any fixed but unknown values in general. In Bayesian approach, you can use probabilities to represent the uncertainty in any event or hypothesis.

# Bayesian and Frequentist statistics

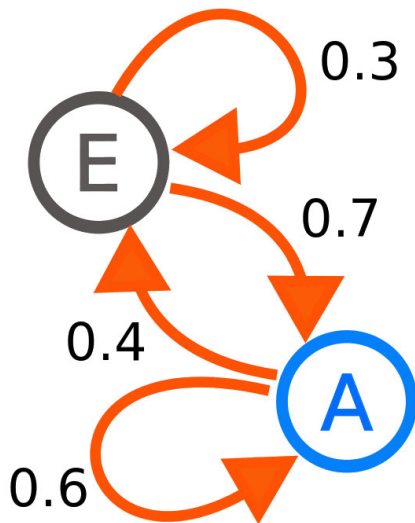
The difference lies in the interpretation of the concept of probability.

In the frequentists' approach, the probabilities are not attached to hypotheses or to any fixed but unknown values in general. In Bayesian approach, you can use probabilities to represent the uncertainty in any event or hypothesis.

**Example:** in Bayesian statistics, you can attach a probability to a non-repeatable event such as Joe Biden becoming the president of USA in 2020. In the frequentists' statistics, you can't do that since you don't have a *statistics* of multiple outcomes of elections in USA in 2020 (you observed just one result!).

# Marcov process and Marcov chain

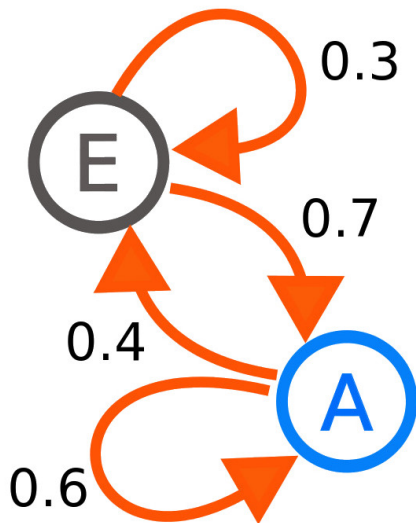
**Marcov process:** a random process with a Marcov property (each random sample is generated based only on the value of the one previous sample)



# Marcov process and Marcov chain

**Marcov process:** a random process with a Marcov property (each random sample is generated based only on the value of the one previous sample)

**Marcov chain:** a “chain” of values generated by a Marcov process (each value depends only on the previous one)



## Example of a Markov model

Many stochastic processes can be described by a **Markov model** (cognitive models, signal detection theory models, market models, etc).

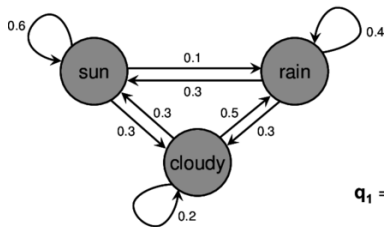
**Most weather models** are very complicated Markov processes (including the models that estimate the probability of El Nino winters).

## Example of a Markov model

Many stochastic processes can be described by a **Markov model** (cognitive models, signal detection theory models, market models, etc).

**Most weather models** are very complicated Markov processes (including the models that estimate the probability of El Nino winters).

A simple toy model: only the states **S = sunny, cloudy, rainy**



$$\mathbf{P} = \begin{bmatrix} 0.6 & 0.3 & 0.3 \\ 0.1 & 0.4 & 0.5 \\ 0.3 & 0.3 & 0.2 \end{bmatrix} \quad \mathbf{q}_0 = \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix}$$

$$\mathbf{q}_1 = \mathbf{P} \cdot \mathbf{q}_0 = \begin{bmatrix} 0.6 \\ 0.1 \\ 0.3 \end{bmatrix} \quad \mathbf{q}_\infty = \mathbf{P}^\infty \cdot \mathbf{q}_0 = \begin{bmatrix} 0.4286 \\ 0.2987 \\ 0.2727 \end{bmatrix}$$

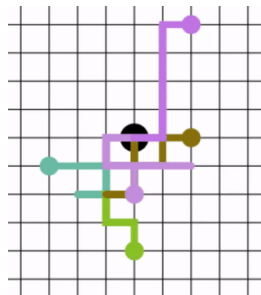
# Properties of Markov chains

- **Irreducible**: a Markov chain is irreducible if you can get from any state to any other state with a non-zero probability in a finite number of steps
- **Aperiodic**: a Markov chain is aperiodic if you can return to any state at any time
- **Ergodic** (or regular): a Markov chain is ergodic if it is **irreducible and aperiodic**

If your process is **ergodic**, you can reach the stationary distribution **no matter the initial distribution**

# Random walk

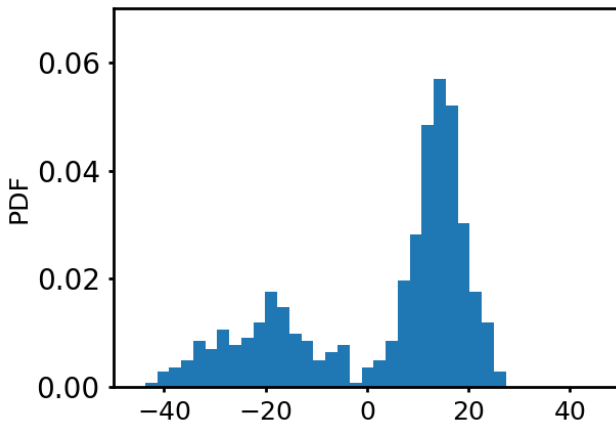
A stochastic process that describes a path that consists of a succession of random steps on some mathematical space such as the integers.



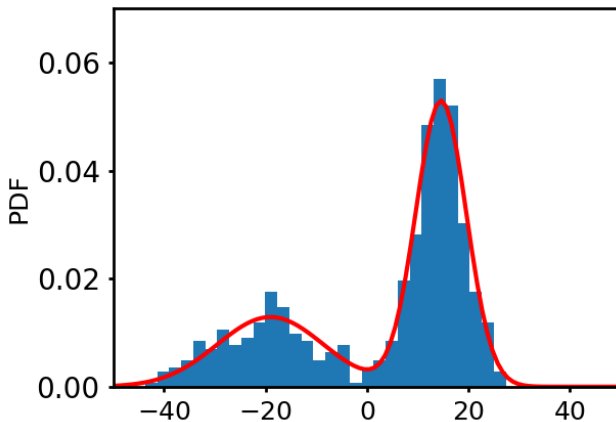
# How do the Bayesian statistics and the Markov processes connect to astrophysical tasks?

- What is the probability that **my model is correct given the observed PDF of the data** (posterior distribution)? Could a different model explain the data better (a higher posterior probability)?
- Given that my model is correct, **what model parameters** are the best to explain the data?
- I have a model **with 100 parameters**. Do I have to run a grid for each one of them, possibly spending months of my server's time, or do I use MCMC instead and obtain a fit quickly?

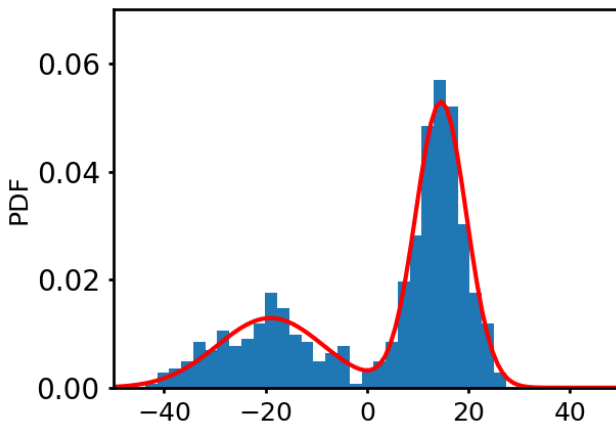
# Probability distribution function (PDF)



## How do I fit the observed PDF?



## How do I fit the observed PDF?



**Wanted:** mean 1 & mean 2 ( $\mu_1$ ,  $\mu_2$ ), standard deviation 1 & 2 ( $\sigma_1$ ,  $\sigma_2$ ) and (optional) the ratio of the peaks ( $R$ ).

## How do I fit the observed PDF?

Since we are dealing with a PDF, the heights of the two peaks (let's call them  $A$  and  $B$ ) and the ratio  $R$  are connected:

$$A = \frac{1}{1 + R}; \quad B = A \cdot R$$

## How do I fit the observed PDF?

Since we are dealing with a PDF, the heights of the two peaks (let's call them  $A$  and  $B$ ) and the ratio  $R$  are connected:

$$A = \frac{1}{1 + R}; \quad B = A \cdot R$$

We need to maximize **the likelihood** which for a given point  $p_i$  looks like this (just like the PDF!):

$$L_i = \frac{A}{\sqrt{2\pi\sigma_1^2}} \exp\left(-\frac{(p_i - \mu_1)^2}{2\sigma_1^2}\right) + \frac{B}{\sqrt{2\pi\sigma_2^2}} \exp\left(-\frac{(p_i - \mu_2)^2}{2\sigma_2^2}\right)$$

## How do I fit the observed PDF?

Since we are dealing with a PDF, the heights of the two peaks (let's call them  $A$  and  $B$ ) and the ratio  $R$  are connected:

$$A = \frac{1}{1 + R}; \quad B = A \cdot R$$

We need to maximize **the likelihood** which for a given point  $p_i$  looks like this (just like the PDF!):

$$L_i = \frac{A}{\sqrt{2\pi\sigma_1^2}} \exp\left(-\frac{(p_i - \mu_1)^2}{2\sigma_1^2}\right) + \frac{B}{\sqrt{2\pi\sigma_2^2}} \exp\left(-\frac{(p_i - \mu_2)^2}{2\sigma_2^2}\right)$$

The **total likelihood**:  $L = \prod_{i=1}^n L_i$  **OR**  $\ln(L) = \sum_{i=1}^n \ln(L_i)$

# How do I fit the observed PDF?

What else do we need?

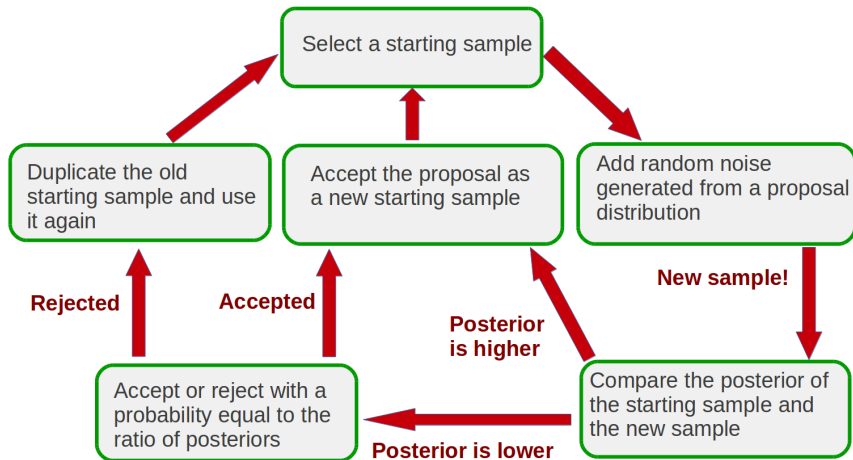
A **proposal**: a new set of parameters for  $\sigma_1, \sigma_2, \mu_1, \mu_2, R$

**Acceptance-rejection** criterion:

$$\alpha = \exp(L_{new} - L_{old})$$

We generate a random number  $r$ : if  $r < \alpha$ , then we **accept the proposal**; otherwise, **we decline**

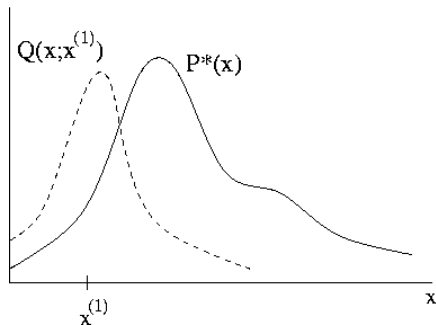
# Metropolis-Hastings algorithm



Repeat until the chain converges

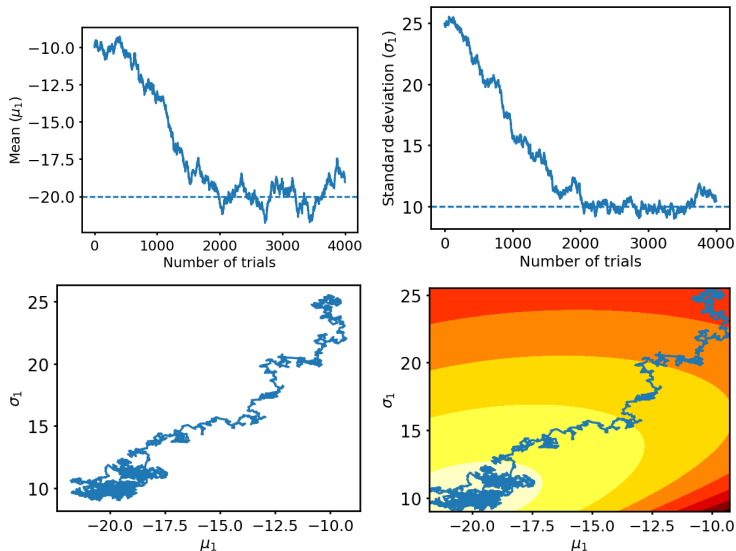
# Metropolis-Hastings algorithm's version of the random walk

The **proposal distribution**  $Q$  proposes the next point to which the random walk might move

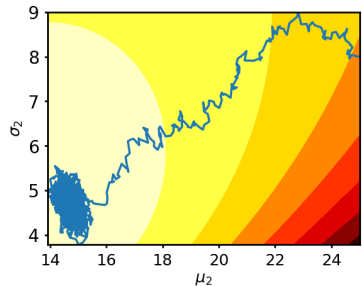
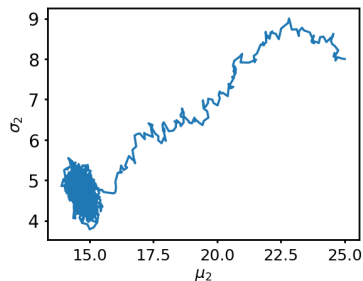
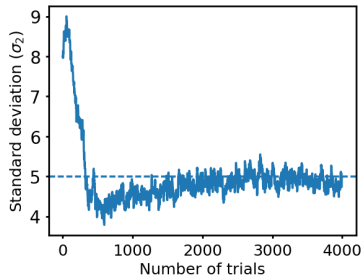
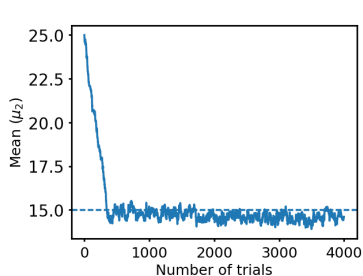


Ideally, the standard deviation of your random step should be chosen so that  $1/e$  proposals are accepted.

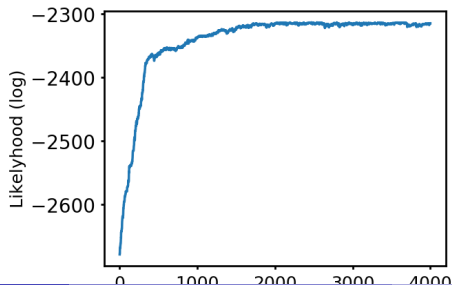
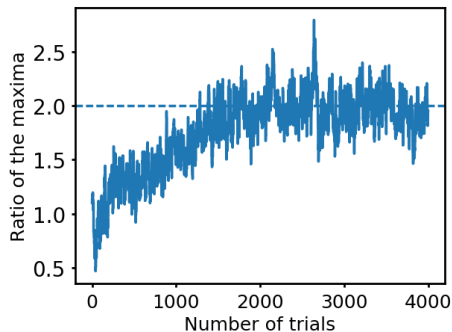
# Marcov chains for $\mu_1$ and $\sigma_1$ (convergence & burn-in)



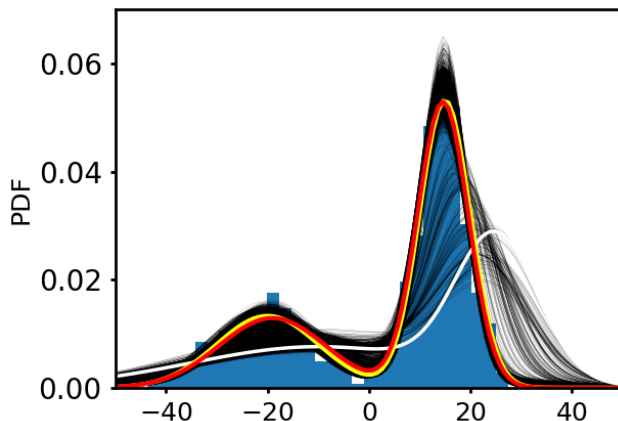
# Markov chains for $\mu_2$ and $\sigma_2$



# Evolution of the likelihood and the maxima ratio



## The final fit and the intermediate steps

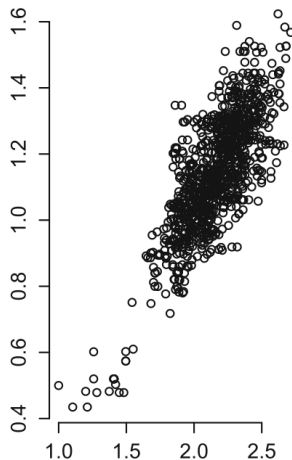


White: initial distribution. Yellow: ideal distribution (was used to generate the data). Red: best fit. Black: all intermediate distributions

# Correlated parameters

What if  $\mu_1$  and  $\sigma_1$  (or any other parameters) in the previous example were correlated?

Correlations between parameters can lead to **extremely slow convergence** of sampling chains, and sometimes to **nonconvergence**.



# Gibbs sampling

Drawing samples for each parameter directly from that parameter's conditional distribution, or the probability distribution of a parameter given a specific value of another parameter. Works well for **strongly correlated parameters**.

In the Gibbs sampling, you generate proposals for all parameters just like in usual MH. However, you accept the new proposal for one parameter only if it is more plausible **given the present value of the other parameter**.

# Gibbs sampling

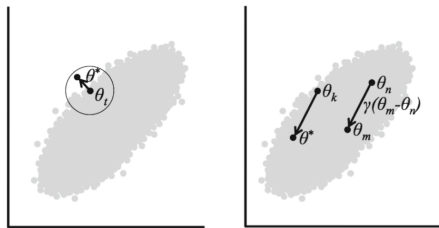
- 1 Generate starting values for your parameters (e.g.,  $A$  and  $B$ )
- 2 Create a new proposal ( $A'$ ) like in the classical MH scheme
- 3 Accept the new proposal if it is more plausible given the  $B$  value (present-one)
- 4 Generate a new proposal for the other variable,  $B'$  from a different proposal distribution. Accept only if it is more plausible given the  $A$  value (or  $A'$ , if the proposal has been accepted in the previous step)
- 5 Return to step 2

...repeat until the chains converge

# Differential evolution (DE): another solution to correlated parameters

Proposals are generated by information **borrowed from the other chains**.

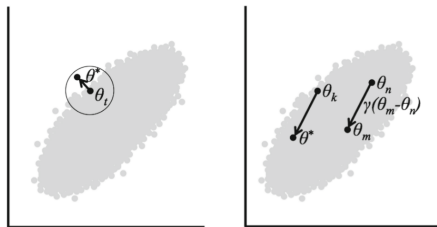
The chains in this examples are:  $\theta_1, \theta_2, \dots, \theta_m \dots$



van Ravenzwaaij et al. 2018

## Differential evolution (DE): another solution to correlated parameters

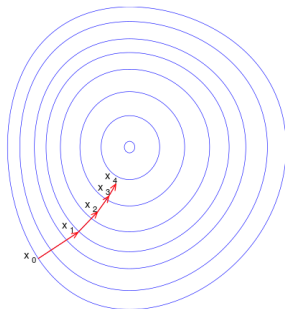
- 1 To get a proposal  $\theta_k$ , choose two random other chains  $\theta_n$  and  $\theta_m$ . Find the distance between their current samples:  $\theta_m - \theta_n$ .
- 2 Multiply  $\theta_m - \theta_n$  by  $\gamma$ . Create the new proposal by adding this to the current sample:  $\theta_k + \gamma(\theta_m - \theta_n)$ . The value  $\gamma$  is a tuning parameter of the DE algorithm.
- 3 Add a very small amount of random noise to avoid identical samples (“degeneracy”)  $\rightarrow$  the new proposal value  $\theta^*$ .



# Other ways of fitting parameters: the gradient decent

The method is good to find the local minimum of a function, **if you have only one parameter to fit**.

MCMC is usually an overkill if you want to fit just one parameter.



To find a **local minimum** of a function, one takes steps proportional to the negative of the gradient of the function at the current point. If one takes steps proportional to the positive of the gradient, one approaches a **local maximum** of that function (gradient ascent).

# Other ways of fitting parameters: acceptance-rejection sampling

Acceptance-rejection sampling (you are interested in  $f(x)$ ):

- 1 Generate a sample  $x$  from a distribution that has a density proportional to  $g(x)$ , where  $g(x) \geq f(x)$  for all  $x$ .
- 2 Generate a  $[0,1)$ -uniform random number  $u$ . If  $u > f(x)/g(x)$  reject the sample and return to Step 1; otherwise accept  $x$  as a sample.

The trick is to choose a proper function  $g(x)$ !

# Other ways of fitting parameters: acceptance-rejection sampling

**Adaptive rejection sampling:** if the PDF is log-concave, i.e.,

$$p(k)^2 \geq p(k-1)p(k+1),$$

we can find a tangential to the function  $g(k) = \log p(k)$  that will always lie above the function  $g(k) \Rightarrow$  this will find the maximum of the PDF and can be used to constrain the rejection sampling

## Other ways of fitting parameters: maximum likelihood estimation (MLE)

A given set of observations is a random sample from an unknown population. MLE: Make inferences about the population that is most likely to have generated the sample (i.e., find a maximum of the function that describes your likelihood).

To maximize the likelihood function, one has to choose appropriate parameters.

## Existing solutions: PyMC3

PyMC3 is a Python package for [Bayesian statistical modeling and Probabilistic Machine Learning](#) focusing on advanced Markov chain Monte Carlo (MCMC) and variational inference (VI) algorithms. Its flexibility and extensibility make it applicable to a large suite of problems.

⇒ a wide variety of MCMC algorithms adapted for different tasks, including the Metropolis-Hastings scheme

# Direct Simulation Monte Carlo

# Outline

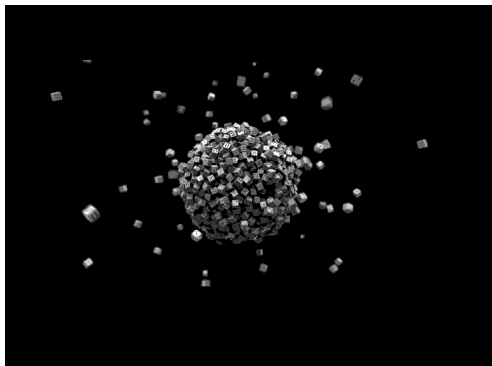
- The idea of Direct Simulation Monte Carlo (DSMC)
- Binary elastic collisions and hard sphere approximation
- Some elements of kinetic theory
- Programming related questions: grids, errors, etc.

Literature on DSMC:

**G.A. Bird**, Molecular gas dynamics and the Direct Simulation of Gas Flows (Caledon Press, Oxford, 1994)

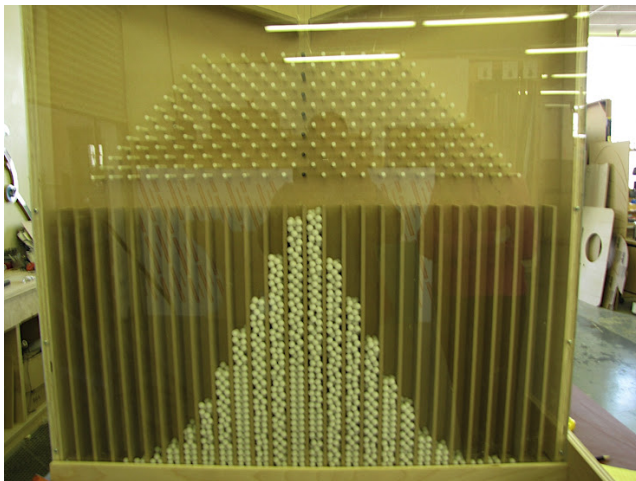
**A. Garcia**, Numerical methods for physics (New Jersey, 2000)

# What is Direct Simulation Monte Carlo?



Tracing of individual particles based on **probability distribution functions (PDFs)**

# How does DSMC work?



An individual particle has **random** parameters. A big ensemble of particles allows to draw conclusions about **macroscopic** parameters.

# Monte Carlo Method

## Advantages:

- Possibility to model difficult processes by easy means
- Almost no restrictions for geometry
- Many simultaneous processes are easy to include

# Monte Carlo Method

## Advantages:

- Possibility to model difficult processes by easy means
- Almost no restrictions for geometry
- Many simultaneous processes are easy to include

## Disadvantages:

- High computational costs
- Weight problems (difficult to model species with different concentrations)
- Long time for conversion

# Monte Carlo Method

## Advantages:

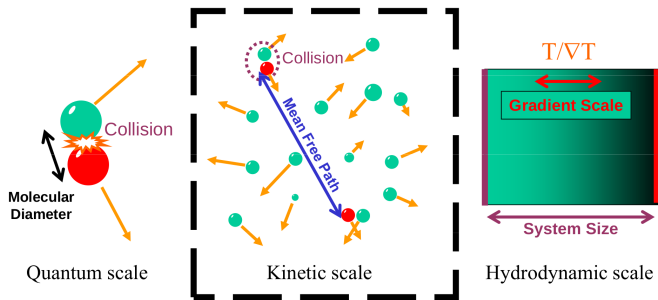
- Possibility to model difficult processes by easy means
- Almost no restrictions for geometry
- Many simultaneous processes are easy to include

## Disadvantages:

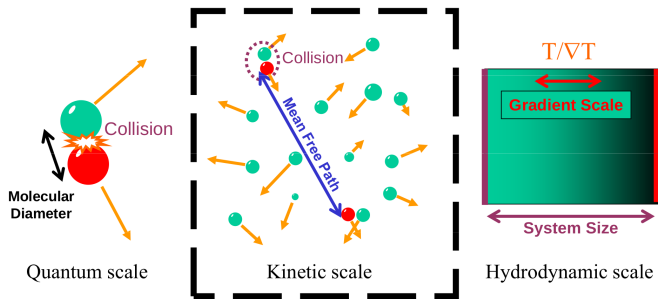
- High computational costs
- Weight problems (difficult to model species with different concentrations)
- Long time for conversion

Under some conditions, DSMC is the only suitable mean for a numerical solution of a problem.

# Knudsen number and the mean free path



# Knudsen number and the mean free path



The degree of rarefaction of the gas is given by the **Knudsen number**, which is the ratio of the **mean free path** of the particles,  $\lambda$ , to the **characteristic dimension**,  $L$  (globally). Or, alternatively, it can be defined (locally) through the gradient of density  $\rho$ :

$$K = \frac{\lambda}{L}$$

$$K = \frac{\rho}{d\rho/dx}$$

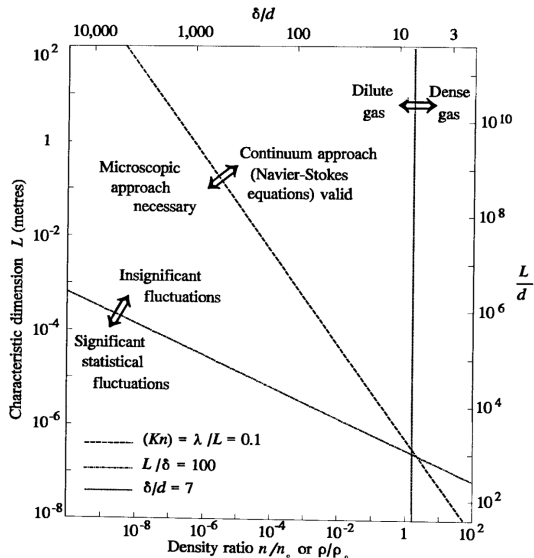
# Choose your method depending on your task

$L$  - length of the simulation domain;

$\delta$  - molecular/particle spacing;

$d$  - effective molecular radius.

G. A. Bird, page 21



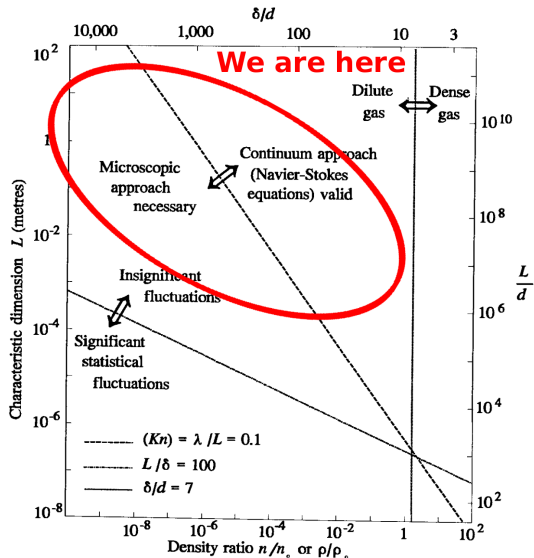
# Choose your method depending on your task

$L$  - length of the simulation domain;

$\delta$  - molecular/particle spacing;

$d$  - effective molecular radius.

G. A. Bird, page 21



# Algorithm

DSMC - a stochastic method for numerical integration

- Generate  $N$  random "points" (particles) in the problem space (each particle has coordinates, velocity, frequency, .., etc)

# Algorithm

DSMC - a stochastic method for numerical integration

- Generate  $N$  random "points" (particles) in the problem space (each particle has coordinates, velocity, frequency, .., etc)
- Advance particles (apply gravity, electromagnetic, other forces, etc, and change the coordinates)

# Algorithm

DSMC - a stochastic method for numerical integration

- Generate  $N$  random "points" (particles) in the problem space (each particle has coordinates, velocity, frequency, .., etc)
- Advance particles (apply gravity, electromagnetic, other forces, etc, and change the coordinates)
- If the particles are in one cell and the probabilistic conditions for an interaction are met, make them interact and change the parameters (possible interactions: elastic collisions, scattering, charge exchange, ionization, recombination, etc)

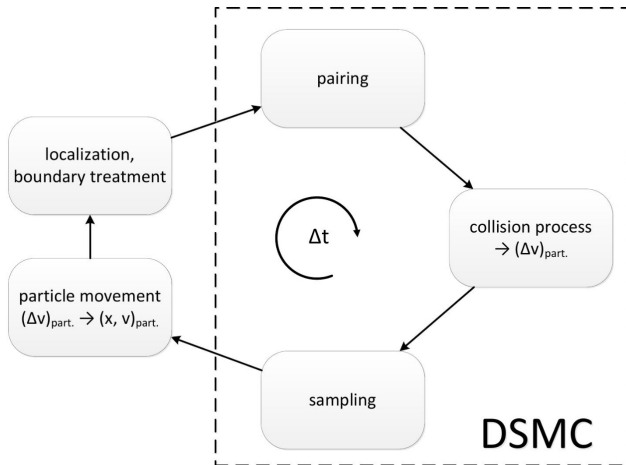
# Algorithm

DSMC - a stochastic method for numerical integration

- Generate  $N$  random "points" (particles) in the problem space (each particle has coordinates, velocity, frequency, ..., etc)
- Advance particles (apply gravity, electromagnetic, other forces, etc, and change the coordinates)
- If the particles are in one cell and the probabilistic conditions for an interaction are met, make them interact and change the parameters (possible interactions: elastic collisions, scattering, charge exchange, ionization, recombination, etc)

According to the Central Limit Theorem, for large  $N$  the parameters will approach the true value

# Algorithm



...plus check every timestep if the number of iteration is **above the maximum one**, if yes - save the data and exit.

# Initialization step

- Create the mesh (cells) and generate particles in the desired cells (e.g., launch atmospheric particles at a sphere)
- For each particle, draw the parameters from the random distribution (e.g., Maxwellian): velocity, temperature, etc.
- Assign each particle position in a cell, either uniformly or using the density gradient
- Determine the number of particles in a cell,  $N_c$  and the weight of each particle

# Basic kinetic theory

Boltzmann equation can be modeled using DSMC. Navier-Stokes - description of gas as a continuum

$$\frac{\partial}{\partial t}(nf) + \mathbf{v} \frac{\partial}{\partial \mathbf{r}}(nf) + \mathbf{F} \frac{\partial}{\partial \mathbf{c}}(nf) = \int_{-\infty}^{+\infty} \int_0^{4\pi} n^2 (f^* f_1^* - f f_1) v_r \sigma d\Omega d\mathbf{v}_1$$

The establishing of a **macroscopic quantity** by averaging over molecules in an element of volume requires that volume  $V \gg \delta$ , where  $\delta$  is the mean molecular spacing  $\delta = n^{-1/3}$ . **Dilute gas**:  $\delta \gg d$  ( $d$  - particle size).

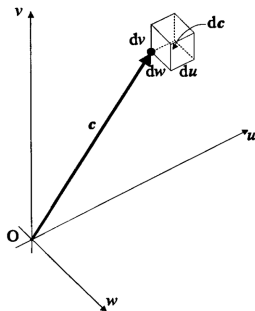
# Velocity distributions

The number  $dN$  gives the amount of particles in a particular part of the **phase space**

$$dN = Nf(c)dudvdw = Nf(c)dc$$

$$\iiint_{-\infty}^{+\infty} f dudvdw = \int_{-\infty}^{+\infty} f dc = N/N = 1$$

The function  $f$  can **never** be negative



# Velocity distributions

The number  $dN$  gives the amount of particles in a particular part of the **phase space**

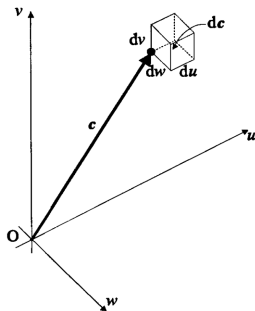
$$dN = Nf(c)dudvdw = Nf(c)dc$$

$$\iiint_{-\infty}^{+\infty} f dudvdw = \int_{-\infty}^{+\infty} f dc = N/N = 1$$

The function  $f$  can **never** be negative

The most common example: **Maxwellian** distribution:

$$f = \left( \frac{m}{2\pi kT} \right)^{3/2} \exp \frac{-mc^2}{2kT}$$



## Example: drawing from a Maxwellian distribution

Calculate approximate **distribution width ( $dw$ )** based on particles mass  $m$  and temperature  $T$

$$dw = \sqrt{\frac{2k_B T}{m}}$$

Then, calculate three **random numbers** distributed according to a Gaussian distribution (e.g. Box-Mueller algorithm) and construct a unit vector  $\mathbf{v}$  from them. Final step:

$$\mathbf{v} = \mathbf{v} * dw / \sqrt{2}$$

If necessary, add the **bulk velocity** of the gas  $\mathbf{v}_{bulk}$  to  $\mathbf{v}$ . Your single particle velocity is ready! Repeat several million times.

# Weight of the particles and collision rate

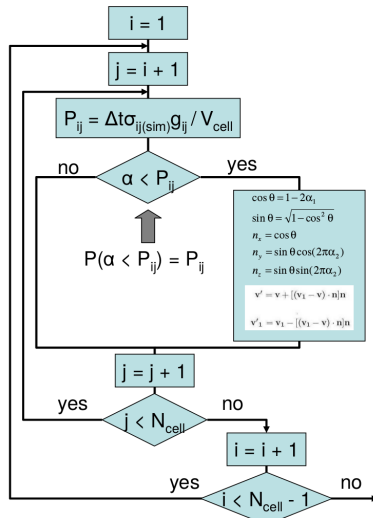
**NB:** In DSMC simulations the number of simulated particles can not be equal to the number of particles in real flow, thus, every “metaparticle” has a specific “weight”,  $N_m$ . Since  $\sigma_1 n_1 = \sigma_2 n_2$ , the simulation cross section should be given as:

$$\sigma_{sim} = \sigma_{real} \frac{n_{real}}{n_{sim}} = \sigma_{real} N_m$$

# Collision sampling: simple scheme

- Calculation of the collision probability between molecules  $i$  and  $j$  ( $g_{ij} = v_1 - v_2$ )
- Does collision between  $i$  and  $j$  occur?
- Calculation of velocities after collision
- Are there other pairs in the cell (if not, go to the next cell)

**Disadvantage** of the primitive scheme: number of operations  $\sim N_c^2$



## Collision sampling: avoiding $N_c^2$

The **total collision frequency**  $\nu_c$  in a volume:

$$\frac{1}{2} n \nu_c = \frac{1}{2} n^2 \overline{\sigma v_r}.$$

For each cell,  $n$  is estimated by  $N_c N_m / V_c$  ( $N_c$  is the number of particles in the cell,  $N_m$  particle weight and  $V_c$  the cell volume)

## Collision sampling: avoiding $N_c^2$

The **total collision frequency**  $\nu_c$  in a volume:

$$\frac{1}{2}n\nu_c = \frac{1}{2}n^2\overline{\sigma v_r}.$$

For each cell,  $n$  is estimated by  $N_c N_m / V_c$  ( $N_c$  is the number of particles in the cell,  $N_m$  particle weight and  $V_c$  the cell volume)

If averages are computed directly: **computational time**  $\sim 1/2 N_c^2$ .

## Collision sampling: avoiding $N_c^2$

The **total collision frequency**  $\nu_c$  in a volume:

$$\frac{1}{2} n \nu_c = \frac{1}{2} n^2 \overline{\sigma v_r}. \quad (1)$$

For each cell,  $n$  is estimated by  $N_c N_m / V_c$  ( $N_c$  is the number of particles in the cell,  $N_m$  particle weight and  $V_c$  the cell volume)

**Possible solution** (see Garcia, 2000):

Calculate a **maximum value**  $(\sigma v_r)_{\max}$  and use it in (1) to compute the number of **trials**. For each trial a random pair is taken, and a random number  $N$ , in the interval  $[0, (\sigma v_r)_{\max}]$ , is computed. If  $(\sigma v_r)_{\max} > N$  for the chosen pair, the **collision is accepted**.

The random pair above is uniformly distributed **if  $N_m$  is the same for all particles**.

## More about collision sampling

**Collision probability** for a pair of particles (Garcia, 2000, p.359):

$$P_{ij} = \frac{|v_i - v_j|}{\sum_{m=1}^{N_c} \sum_{n=1}^{m-1} |v_m - v_n|}$$

Instead of computing it directly, **acceptance-rejection** scheme is used:

- 1 Select random particles  $i$  and  $j$
- 2 Calculate  $v_r = |v_i - v_j|$
- 3 Accept the collision and process the velocities of the particles, if  $v_r > v_r^{max} R$ , where  $v_r^{max}$  is the maximum relative velocity in the cell and  $R \in [0, 1)$
- 4 Return to step 1

# Random numbers

Be sure to use **high quality** random generator!

**Drawing from a random distribution** (for a Gaussian, exponential distributions, etc): Generate random value  $x$  with distribution  $P(x)$  as  $x = f(R)$  where  $R$  is uniformly distributed in  $(0,1)$ .

**Accept-reject method** (for most other distributions): Draw  $x_{try}$  uniformly in the range of  $x$ ; accept it if  $P(x_{try}) > \max P(x)R$ , else draw again.

# Simple domain boundaries

- **Periodic boundaries:** particles are reintroduced on the other side of the system
- **Specular surfaces:** particles are reflected at the boundary
- **Reservoirs boundary:** boundary treated as a reservoir with particles with given density, temperature, etc. Particles in the main system are removed if they cross the boundary into the reservoir

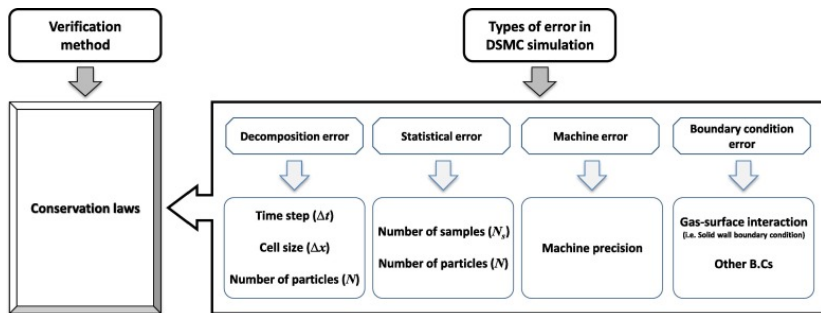
# Simple domain boundaries

- **Periodic boundaries:** particles are reintroduced on the other side of the system
- **Specular surfaces:** particles are reflected at the boundary
- **Reservoir boundary:** boundary treated as a reservoir with particles with given density, temperature, etc. Particles in the main system are removed if they cross the boundary into the reservoir

**Injection of particles** from the reservoir to the main system:

- **Surface generator:** from number flux determines number to be injected; generate particle velocities from surface distribution (e.g., Maxwellian)
- **Volume generator:** create a “ghost cell” with particles before the ballistic move; discard any that do not enter the main system during the move phase.

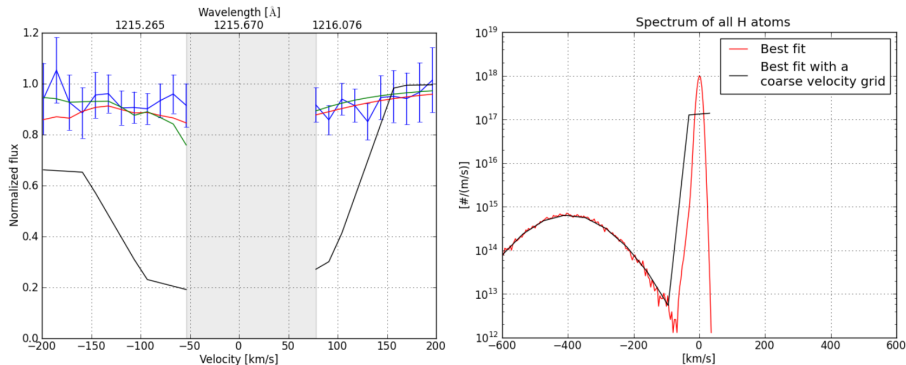
# Cell size and time step



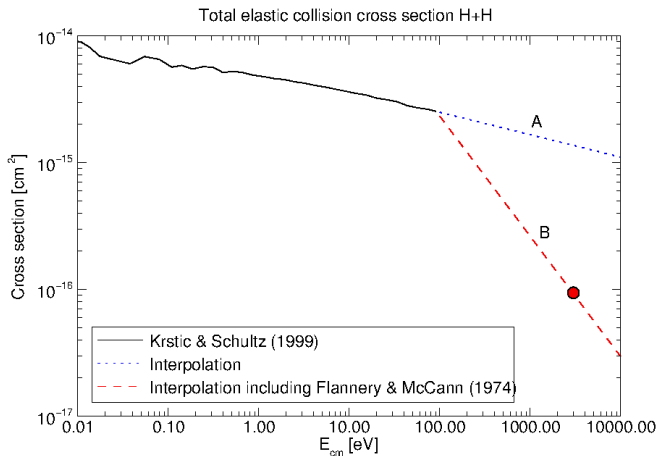
$L/\delta = 1000$  (to avoid significant fluctuations; Bird, page 22);  $\delta$  should be much larger than mean de Broglie wavelength of particles (page 23) to avoid quantum effects

# Discretization errors

An example of an error introduced by a **coarse grid** (in this case, velocity grid). **Something to be careful about!**

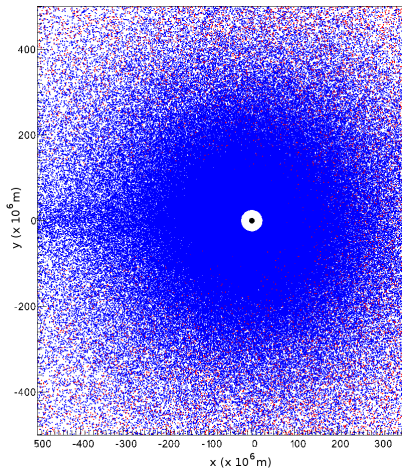


# Cross sections problems

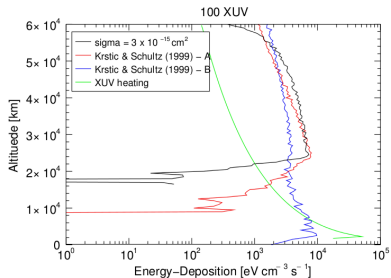
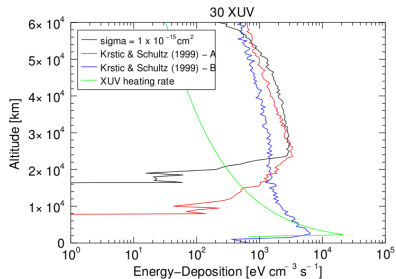


Lichtenegger+, 2016

# Cross section: influence on results



Lichtenegger+, JGR, 2016



# Simulation of atmospheres

DSMC simulation 1 (click for video)

DSMC simulation 2 (click for video)

# Summary

- The MCMC methods are a good mean to fit multiple parameters at once, which gives them an advantage compared to simple methods such as the gradient descent
- In both DSMC and MCMC, one should always be careful about convergence towards the stationary distribution: sometimes your code might not converge
- The correlation between samples can be a problem, so a sufficient amount of samples (steps) is necessary
- The average size of the step should be chosen carefully